5

PART 1: FOUNDATION

Assembling the Specification

Four documents exist for the Riverside Clinics booking feature by the time this chapter opens, and each one is good. The vision, written with the front desk lead and a nurse in the room, names the patients who skip care entirely when booking means saying the reason out loud. The requirement from the workshop pins the booking horizon down to ninety days, a number a test can check. The Schedule Appointment use case runs the whole interaction start to finish, alternative flows included. The domain model gives Patient, Doctor, Clinic, and Appointment their attributes and the rules connecting them. Written separately, reviewed separately, each one earned its place in the last three chapters.

Fragmented Specifications

They also live in four different places. The vision is a paragraph from a stakeholder meeting, still sitting in someone's notes. The requirement is a line item in a tracker. The use case is a document a business analyst wrote and a developer skimmed. The domain model is a diagram an architect drew on a whiteboard and photographed. When a developer sits down to implement the feature, she opens the AI assistant and pastes in the use case, because that's the document describing what the code has to do. She doesn't attach the domain model; it lives in a different tool. She doesn't attach the ninety-day requirement either; she has seen it before and assumes the use case already covers it. On that one she is right, because the use case carries the ninety-day rule in its business rules.

The code that comes back handles the happy path correctly and misses the fact that decides whether a booking is usable at all: a Riverside doctor works at more than one clinic, so a booking has to name a clinic where that doctor actually works. The use case she pasted doesn't say that. It lived one document over, in the domain model she didn't attach.

The mistake in the last three chapters was missing information: no stated reason for the system, no requirement precise enough to test, no model of what a doctor and a patient are to the business. This chapter's mistake is different. Every piece of information exists. It simply isn't in the same room.

An AI assistant, like a new hire, implements from what it is handed. Hand it a complete use case and nothing else, and it will implement that use case correctly, on its own terms. It has no way of knowing that a business rule belonging to the use case lives in a domain model two documents away, because from where it sits, that document does not exist. The resulting code follows the instructions it was given, to the letter. Those instructions were incomplete because no one had reassembled the specification before handing it over, even though every piece of it already existed.

The review that follows looks identical to the one in Chapter 1: a developer finds the gap, traces it back, and either patches the code or goes looking for the missing piece. The hours lost are the same hours. The cause is different, and harder to spot, because the second one hides behind documents that all, individually, look finished.

Purpose of an Assembled Specification

Each layer built in Chapters 2 through 4 answers a question the others don't. The vision answers why the system exists and for whom. The requirement answers what specific, observable behavior it must have. The use case answers what a complete interaction looks like, start to finish. The domain model answers what business objects the system runs on, and how they relate. None of the four can stand in for another. A vision that lists database fields has stopped being a vision. A domain model that describes user steps has stopped being a domain model.

A complete specification places its four documents in agreement with each other, in one place, before anyone, human or AI, starts implementing from any single one of them. Agreement here means something specific: the same term names the same thing everywhere it appears. If the vision names a rule about doctors and locations, that rule has to survive into the requirement as something testable, into the use case as a step or a business rule, and into the domain model as a relationship between two entities. A rule that appears in only one of the four layers is a rule your AI assistant probably won't see, because what it reads is what it was handed.

> **A specification is complete when its four layers agree with each other in one place, not when each layer merely exists.**

Assembling the Four Layers

1. Place the vision at the top, in the words the stakeholders agreed to, not restated by whoever is assembling the document. A reader should meet the reason before the rule.
2. Attach only the requirements this feature's use case depends on, not the full backlog. A ninety-day booking window belongs here. An unrelated requirement about billing does not.
3. Add the use case in full: actor, preconditions, main scenario, alternative flows, postconditions, business rules. This is the shape an AI assistant implements best, and summarizing it defeats the reason for including it.
4. Add the domain model, limited to the entities and attributes the use case names. Then check each one against the use case by name; a use case that says "Doctor" and a model that says "Physician" are describing the same thing to a person and two different things to an AI assistant.
5. Version the whole assembly as one document, dated. When a requirement changes, this is the one file that changes, not four scattered guesses reconciled after the fact.

Worked Example: The Riverside Clinics Specification

Here is what steps one and two produce for Riverside Clinics: the vision from Chapter 2, condensed, with the multi-location fact from Chapter 4's model added to it, followed by the two requirements this use case depends on most directly, the rule that turns "available" into something a system can check rather than something a receptionist remembers, and the rule that keeps the reason for a visit from traveling further than the people delivering the care.

Specification: Schedule Appointment, Riverside Clinics (v1.0)

1. VISION

Patients book by phone or in person at the front desk. A doctor works at one or more clinics, and which doctor is at which clinic on which day is knowledge only the front desk holds, so a patient booked at the wrong clinic arrives and is turned away. Both routes also require stating the reason for a visit where staff, and sometimes other patients, can hear it. Some patients tolerate that. Others avoid booking entirely rather than disclose it aloud, and the clinic never learns they needed care. We want patients to book, reschedule, and cancel appointments online, privately. This system is not responsible for billing, medical records, or clinical intake, which stay with the systems that already handle them.

2. REQUIREMENTS

REQ-07 (availability): A time slot is available for booking only if all three hold: it falls within 90 days of the current date; the doctor works at that clinic; and no existing appointment already occupies it. The system must not display a slot to a patient unless all three are true, and must state which condition failed when a patient requests one that isn't.

REQ-08 (confidentiality): A patient's stated reason for the visit is visible only to that patient, the doctor named on the appointment, and staff at the clinic named on the appointment. The system must not show it to staff at Riverside's other clinics, and confirmations, reminders, and cancellation notices must name the doctor, date, time, and clinic without naming the reason.

Worked Example, Continued: The Use Case

The use case below is Chapter 3's, condensed, with the clinic named where the domain model says it belongs. In the working document it sits at full length, every step and every alternative flow written out.

> **Specification: Schedule Appointment, Riverside Clinics (continued)**
>
> 3. USE CASE: Schedule Appointment
>
> Actor: registered patient, or a receptionist booking on the patient's behalf. Preconditions: patient is logged in; at least one doctor is accepting bookings within the request window.
>
> Main scenario: (1) patient selects a clinic or a doctor; (2) system displays only slots satisfying REQ-07; (3) patient selects a slot; (4) system reserves the slot and creates the appointment record; (5) system sends confirmation naming the doctor, date, time, and clinic.
>
> Alternative flows: if no slot satisfies all three conditions, the system offers the same doctor at a later date or another doctor with availability in the requested range, rather than a bare "not available." If a slot is taken between steps 2 and 4, the system notifies the patient and redisplays current availability.
>
> Postconditions: the appointment exists, tied to one patient, one doctor, one clinic, one time slot; confirmation has been sent.
>
> Business rules: slots run fifteen minutes, inside clinic hours; cancellation requires at least 24 hours' notice, otherwise the patient calls the clinic; doctor availability is entered by clinic staff, not pulled from an external calendar.

Worked Example, Concluded: The Domain Model

> **Specification: Schedule Appointment, Riverside Clinics (concluded)**
>
> ```
> 4. DOMAIN MODEL (entities the use case above names)
> ```
>
> ```
> Patient: a person who receives care; has many Appointments. Doctor: works at
> one or more Clinics, many-to-many, the relationship this whole feature turns
> on; has many Appointments. Clinic: one physical Riverside location; employs
> many Doctors, handles many Appointments. Appointment: belongs to one Patient,
> one Doctor, one Clinic, one TimeSlot; status is Scheduled, Completed,
> Cancelled, or NoShow. TimeSlot (value object): date, start time, end time; has
> no identity of its own outside an Appointment.
> ```

Read start to finish, one fact threads through all four sections in the same words. What the vision records as a doctor working at one or more clinics becomes the second condition in REQ-07, becomes the filter the use case applies before it shows a patient anything, and becomes the many-to-many Doctor-Clinic relationship in the domain model. An AI assistant implementing from this one document doesn't have to go looking for that connection. It has already been made.

Common Assembly Mistakes

▲ **WARNING**

Assembling the four layers once is not the same as keeping them assembled. A specification that stops changing the day the feature ships decays the same way any other documentation decays, unnoticed until it no longer describes what is running.

Treating assembly as a one-time step	Six months after launch, a rule changes in production and nobody updates the assembled document. New work against the feature reverts to guessing, because the one place that was supposed to hold the truth no longer does.
Assembling everything instead of what one use case needs	Pulling in the full domain model and the entire requirements backlog produces a document long enough that reviewers stop reading it, which defeats the reason it was assembled in the first place.

Template: One-Page Specification

Six sections, one file. The note against each says what belongs there. It doesn't tell you how to word it.

- ☐ Header: feature name, version number, date. One line, so a reader knows which assembly they are holding and which one it replaced.
- ☐ 1. Vision: why this exists and for whom, in the words the stakeholders agreed to.
- ☐ 2. Requirements: only the ones this use case depends on, each written as something a test can check.
- ☐ 3. Use case: actor, preconditions, main scenario, alternative flows, postconditions, business rules. All six lines filled.
- ☐ 4. Domain model: the entities, attributes, and relationships the use case names, under the names the use case uses.
- ☐ 5. Sign-off: the date the four sections above were last cross-checked against each other, so a reader knows how current the agreement between them is.

Fill every section before handing the document to an AI assistant or a new team member. The sign-off date looks like bookkeeping. It is the only line that tells a reader whether the four sections above it still agree with each other.

Summary

- A specification is complete when its four layers exist together and agree with each other, not when each one merely exists somewhere.
- Assembling costs time up front. That cost is repaid the first time a requirement changes, since one document gets updated instead of four that then drift apart.
- The assembled specification, not the code generated from it, is what should still be correct after the code has been rewritten twice.
- Consistency between layers, the same term meaning the same thing everywhere it appears, matters more than how complete any single layer is on its own.

DISCUSSION QUESTIONS

- Where do this feature's vision, requirement, use case, and domain model live right now, and how many places is that?
- The last time a requirement changed, what else changed with it, and who was responsible for each piece?

PRACTICAL EXERCISE

Take a feature currently in flight. Gather whatever exists for it, vision, requirements, use case, domain model, even if some of it is incomplete, and assemble it into one document today. Cross-check it once against the one-page template on the previous page before anyone implements another line against it.

Assembling the Specification ends here.